

Mezcla óptima de pronósticos

Planteamiento y solución computacional

Bryan Rodríguez Abarca · 16 de agosto de 2026

1. Planteamiento

Un área de planeación con cuatro modelos de pronóstico de demanda diaria suele reportarlos por separado. Sobre la ventana de validación de 240 días de este trabajo, promediarlos a partes iguales deja un error cuadrático medio de 276.38 unidades² y el mejor por su cuenta se queda en 426.76. Si repartir el peso a ciegas ya gana tanto, la pregunta es cuál es el reparto que menos error deja.

El reparto no puede ser cualquiera: la mezcla se comunica como «tanto por ciento de cada modelo», así que los pesos suman uno y ninguno puede ser negativo; un peso negativo equivale a restar un pronóstico y saca la mezcla del rango de los que la alimentan.

Los datos son sintéticos y se generan en el cuaderno adjunto con la semilla 1185: una serie con estacionalidad semanal, tendencia y ruido, y cuatro pronósticos con estructuras de error deliberadamente distintas — A con ruido propio, B con la mitad del error de A más ruido independiente, C con error independiente y más grande, y D, la variante reducida de un modelo ya en producción, que reproduce el error de A multiplicado por 1.15 con poco ruido encima: la correlación entre los errores de A y de D es 0.9436.

2. Justificación y modelo matemático

Lo que se decide es el peso de cada modelo, y nada más: la matriz de pronósticos P (240 filas por 4 columnas) y la demanda observada y ya están fijas cuando empieza la decisión, así que el error depende solo del vector de pesos y la pregunta de negocio queda como un problema de optimización sobre cuatro números, con límites que salen del formato en que se reporta la mezcla.

Variables de decisión: $w_1, \dots, w_4$, adimensionales, la fracción de la mezcla que aporta cada modelo.

Función objetivo: el error cuadrático medio del pronóstico combinado, en unidades² de demanda diaria, que se minimiza. **Restricciones:** igualdad $\sum_i w_i = 1$ y cotas $0 \leq w_i \leq 1$ para cada i .

$$\text{ECM}(w) = \frac{1}{n} \|Pw - y\|^2 = w^\top Gw - 2c^\top w + \overline{y^2}, \quad G = \frac{1}{n} P^\top P, \quad c = \frac{1}{n} P^\top y.$$

La forma cuadrática deja el gradiente exacto a la vista, $\nabla \text{ECM}(w) = 2(Gw - c)$, y clasifica el problema. Es continuo, no lineal — cuadrático en los pesos —, con restricciones, y convexo, porque G es una matriz de Gram y por tanto semidefinida positiva, y la región factible es la intersección de un hiperplano con una caja. En un problema convexo todo mínimo local es global (Nocedal y Wright, 2006, cap. 12), así que el punto inicial no decide el óptimo y el resultado se comprueba con las condiciones de optimalidad. De los métodos que documenta SciPy (The SciPy community, 2024), **SLSQP** admite igualdades y cotas a la vez; **trust-constr** entra después como control de precisión.

3. Código Python documentado

El cuaderno adjunto (Python 3.9.6 · numpy 2.0.2 · scipy 1.13.1, semillas 1185 y 2026) va en diez bloques; aquí quedan condensados los que definen el modelo y lo resuelven.

```
G = P.T @ P / DIAS
c = P.T @ y / DIAS

def ecm(w):
    # error cuadrático medio, unidades^2
    w = np.asarray(w, dtype=float)
    return float(np.mean((P @ w - y) ** 2))
```

```
def grad_ecm(w):
    # gradiente exacto: 2 (G w - c)
    return 2 * (G @ np.asarray(w, dtype=float) - c)
suma_uno = {"type": "eq", "fun": lambda w: np.sum(w) - 1.0}
limites = [(0.0, 1.0)] * 4
w0 = np.full(4, 0.25) # el promedio simple: factible y neutral
res = minimize(ecm, w0, method="SLSQP", jac=grad_ecm,
               bounds=limites, constraints=[suma_uno],
               options={"ftol": 1e-12, "maxiter": 300})
```

La igualdad mezcla las cuatro variables y va en **constraints**, con la convención de scipy de valer cero cuando se cumple; las cotas van en **bounds**. El punto inicial es el promedio simple, que además es la alternativa contra la que se compara el resultado. El gradiente exacto cabe en una línea porque el objetivo es cuadrático. La tolerancia en 10^{-12} y medir la demanda en unidades salieron de una corrida que falló.

4. Análisis del resultado

Bloque 4 del cuaderno, sin las líneas de mensaje y de tiempo:

```
exito           : True
iteraciones     : 13
eval. objetivo  : 12
eval. gradiente : 9

peso del modelo A : 0.485196
peso del modelo B : 0.283562
peso del modelo C : 0.231242
peso del modelo D : 0.0
ECM de la mezcla : 256.40479
RECM de la mezcla : 16.0126 unidades/día
```

El modelo D queda fuera, como la construcción de la serie tenía decidido: su error es el de A amplificado, y el peso nulo muestra que el método reconoce la redundancia y le pone precio. La suma de pesos menos uno da cero exacto y el peso de D se queda a 5.15×10^{-13} de su cota, así que la solución es factible. Que **success** sea **True** no dice que además sea óptima, así que lo comprobé por dos caminos ajenos al optimizador. Las condiciones de optimalidad piden que el gradiente sea constante sobre los pesos libres — lo es, con desviación máxima 2.08×10^{-6} — y que el multiplicador de la cota activa sea positivo: el de D vale 55.68, lo que cuesta en error moverle una unidad de peso. El otro camino es una referencia cerrada: con $w_D = 0$ queda una cuadrática con una igualdad lineal, que sale del sistema $\begin{bmatrix} 2G & 1 \\ 1^T & 0 \end{bmatrix} \begin{bmatrix} w \\ -\lambda \end{bmatrix} = \begin{bmatrix} 2c \\ 1 \end{bmatrix}$ sin optimizador de por medio. El resultado numérico se separa de esa referencia 1.07×10^{-9} en el peso peor colocado, y el valor del objetivo coincide con error relativo 1.11×10^{-13} .

4.1. Qué gana la mezcla y qué cuesta la restricción

	ECM (u ²)	RECM (u/día)
Mejor modelo individual (A)	426.7563	20.6581
Promedio simple de los cuatro	276.3815	16.6247
Mezcla óptima con pesos no negativos	256.4048	16.0126
Mezcla óptima sin exigir signo	245.5116	15.6688

Optimizar los pesos ahorra 0.6121 unidades diarias de RECM frente a promediar — un 3.68 %, que remuestreando los 240 días queda en 4.02 con desviación estándar 0.99 — y 4.6454 frente a usar solo el mejor modelo (20.65808 – 16.01264); el promedio ya reparte peso entre errores que se cancelan en parte, de ahí que la ganancia sea modesta. Sin la exigencia de no negatividad, el reparto óptimo sería (0.9149, 0.2614, 0.2149, –0.3913), con D restando la parte del error de A que reproduce amplificada, y vale 245.5116 contra 256.4048. La política de reportar porcentajes cuesta entonces 10.893 unidades², o 0.3438 unidades diarias de RECM.

4.2. ¿Es eficiente?

Método	iter.	eval. f	ms (mediana)	error vs. exacta
SLSQP, gradiente exacto	13	12	1.095	1.07×10^{-9}
SLSQP, diferencias finitas	13	48	1.707	1.52×10^{-8}
trust-constr, gradiente exacto	64	72	29.410	2.97×10^{-13}

La columna de tiempo es la mediana de siete lotes en esta máquina, macOS sobre ARM: los lotes de SLSQP van de 1.091 a 1.106 milisegundos y los de `trust-constr` de 29.273 a 29.526. En otra máquina los absolutos cambian; iteraciones y errores son idénticos en cada ejecución. La vara de precisión es el mismo sistema resuelto a 60 dígitos: la referencia cerrada en `float64` está a 3.400×10^{-13} de ella, y ninguna comparación afirma nada por debajo de su vara. `trust-constr` llega tres órdenes y media más cerca por 26.9 veces más tiempo. Escribir el gradiente a mano bajó las evaluaciones de 48 a 12.

4.3. ¿Qué mejorarías? La corrida que falló

La primera versión medía la demanda en miles, que es como viene el reporte de operaciones. Con esa escala y las opciones por omisión, `minimize` devolvió el promedio simple con el que arrancó, marcó `success = True` y no avisó de nada.

Escala	<code>ftol</code>	iter.	peso de A	peso de D
unidades	10^{-6}	8	0.485187	0.000000
miles	10^{-6}	1	0.250000	0.250000
miles	10^{-8}	7	0.258497	0.148614
miles	10^{-10}	23	0.485203	0.000000

La causa no es el gradiente numérico: repetí la corrida en miles con el gradiente exacto y devolvió el punto inicial, igual a w_0 bit a bit y con un cambio de exactamente cero; nunca dio un paso. Lo que `ftol` atraviesa en la primera iteración es el descenso que promete el paso por dar, salido de un subproblema cuya curvatura aún es la identidad: ese descenso vale el cuadrado de la norma del gradiente proyectado sobre $\sum_i w_i = 1$. Al cuadrado arrastra dos veces el factor de escala y `ftol` ninguno: en unidades vale 1.3367×10^4 y en miles 1.3367×10^{-8} , casi 75 veces por debajo de la tolerancia por omisión. Compensarla pediría dividir la tolerancia entre ese 10^{12} , no entre el 10^6 que separa las escalas, y ni así coinciden las corridas: 10^{-6} en unidades da 0.485187 en ocho iteraciones y 10^{-18} en miles, 0.485196 en veintiséis. Lo compensado es el criterio de paro, no el camino: la curvatura inicial es la identidad en las dos escalas y el gradiente no. El renglón de 10^{-8} en miles es el más incómodo: el método se mueve, se detiene a medio camino con el peso de D en 0.1486 y sigue reportando éxito.

4.4. ¿Es aplicable?

Los pesos ajustados sobre los primeros 160 días, aplicados a los 80 que no vieron, dan 15.1813 unidades diarias de RECM contra 15.8293 del promedio simple y 21.2636 de A solo. Moviendo el corte de 120 a 200 días, la ventaja sobre el promedio va de 0.4730 a 0.6751 unidades diarias, con mediana 0.5782, y no cambia de signo en los nueve cortes.

El reparto entre A y B no aguanta seis decimales. Remuestreando los 240 días 300 veces, el peso de A tiene desviación estándar 0.0454 y su intervalo del percentil 5 al 95 va de 0.4107 a 0.5651; el número de condición de G es 32 590 y las columnas de A y D se hacen sombra. La exclusión de D sí resiste: su peso es cero en los 300 casos y lo defendible no es «A pesa 0.4852» sino «D no entra».

Supuestos por renegociar antes de llevarlo a un proceso real: que la correlación entre los errores se mantenga, que 240 días cubran la estacionalidad, y que el error cuadrático medio sea la pérdida del negocio, cuando pasarse y quedarse corto rara vez cuestan igual.

5. Conclusiones

Apretar el reparto entre A y B, que no está determinado con la precisión que el optimizador imprime, pide una ventana móvil con penalización hacia el promedio simple: trabajo de modelado.

La mezcla óptima reparte el peso entre A, B y C y excluye a D, con 16.0126 unidades diarias de RECM, verificada contra una solución en forma cerrada y las condiciones de optimalidad. Exigir pesos no negativos cuesta 0.3438 unidades diarias: el argumento cuantitativo para discutir la política de reporte.

Referencias

Nocedal, J., & Wright, S. J. (2006). *Numerical optimization* (2nd ed.). Springer.

The SciPy community. (2024). *scipy.optimize.minimize* [Documentación de SciPy 1.13.1]. <https://docs.scipy.org/doc/scipy/reference/optimize.html>